



Pontifícia Universidade Católica do Rio Grande do Sul
Confiabilidade e Segurança de Software
98G08-04
Lista de Exercícios
Funções Hash: Teoria, SHA-2 e SHA-3

Prof. Iaçanã Ianiski Weber

Exercícios adaptados de Paar, Pelzl e Güneysu, *Understanding Cryptography*, 2ª ed., Springer, 2024 (Capítulo 11).

Exercício 1

Sistemas de autenticação costumam armazenar $h(\text{senha})$ em vez da senha: o valor digitado é processado por h e comparado com o hash guardado. Suponha que um atacante obteve o arquivo com todos os hashes de senha. (Livro, Problema 11.5)

- (a) Para cada cenário abaixo, discuta se o ataque permite recuperar senhas para se passar por usuários legítimos e com quais consequências: **Ataque A:** quebrar a propriedade de mão única (*one-wayness*) de h ; **Ataque B:** encontrar segundas pré-imagens de h ; **Ataque C:** encontrar colisões de h .
- (b) Por que essa técnica costuma ser estendida com um *salt* (valor aleatório anexado à senha antes do hash)? Os ataques do item (a) são afetados pelo *salt*?
- (c) Uma função hash com saída de **80 bits** é suficiente para essa aplicação? Justifique com base nas propriedades que ela realmente exige.

Exercício 2

Considere três funções hash com saídas de 64, 128 e 160 bits. O número esperado de entradas aleatórias que precisam ser processadas para que ocorra uma colisão com probabilidade λ é aproximadamente

$$t \approx 2^{(n+1)/2} \sqrt{\ln\left(\frac{1}{1-\lambda}\right)},$$

onde n é o comprimento da saída em bits. (Livro, Problema 11.3)

- (a) Para cada um dos três comprimentos, após quantas entradas aleatórias há probabilidade $\lambda = 0,5$ de colisão?
- (b) E para $\lambda = 0,1$?

Exercício 3

Vamos computar à mão a primeira rodada ($j = 0$) da função de compressão do SHA-256. Ignore os valores iniciais padronizados e assuma todos os registradores zerados: $A_0 = \dots = H_0 = 00000000_{\text{hex}}$. Lembre que $T_1 = H + \Sigma_1(E) + Ch(E, F, G) + K_j + W_j$, $T_2 = \Sigma_0(A) + Maj(A, B, C)$, e que $K_0 = 428A2F98_{\text{hex}}$. (Livro, Problema 11.9)

- (a) Calcule os registradores $A_1, \dots, H_1$ para o bloco $x = \{0 \dots 00\}$ (todas as palavras W_j nulas).
- (b) Repita para $x = \{0 \dots 01\}$, isto é, $W_0 = 00000001_{\text{hex}}$ e as demais palavras nulas.
- (c) O que (a) e (b) sugerem sobre o papel das constantes K_j quando a entrada é toda nula?

Exercício 4

Use uma ferramenta de linha de comando para calcular o hash SHA-512 das mensagens $m_1 = \text{Have you eaten, my child?}$ e $m_2 = \text{Have you eaten my child?}$. Calcule o hash de cada mensagem *sem* o caractere de nova linha ao final; no Linux, por exemplo: `printf 'Have you eaten, my child?'`
| `sha512sum` (Livro, Problema 11.10)

- (a) Apresente os quatro primeiros bytes de cada hash em notação hexadecimal.
- (b) As mensagens m_1 e m_2 são quase idênticas (diferem em uma vírgula). O que você observa nos hashes? Que nome damos a esse comportamento e por que ele é desejável?

Exercício 5

O Keccak- f é uma **função de permutação**: cada um dos 2^b valores possíveis do estado é mapeado, de forma bijetiva (um-para-um), para exatamente um valor de saída. Isso é diferente de uma **permutação de bits** (como as permutações P e IP do DES), que apenas troca as *posições* dos bits. Para enxergar a diferença, considere um exemplo em miniatura com $d = 2$ bits de entrada e saída. (Livro, Problema 11.13)

- (a) Quantas permutações de bits existem com 2 bits de entrada e saída? Desenhe um diagrama para cada uma.
- (b) Quantos valores distintos de entrada (e de saída) existem? E quantas funções de permutação (mapeamentos bijetivos) distintas existem entre entrada e saída? Justifique com um argumento de contagem.
- (c) Dentre as funções de permutação do item (b), quais correspondem às permutações de bits do item (a)? Mostre as duas tabelas-verdade.
- (d) Generalize: para d bits, quantas permutações de bits e quantas funções de permutação existem? O que esses dois números sugerem no caso do estado do Keccak- f ($b = 1600$)?

Exercício 6

Suponha que o SHA-3 é usado em uma aplicação com saída de 256 bits (SHA3-256, taxa $r = 1088$ bits) e que medimos uma vazão de **120 MByte/s** com nossa biblioteca criptográfica. Por razões de segurança, decidimos trocar para o SHA3-384, cuja taxa é $r = 832$ bits. (Livro, Problema 11.11)

- (a) Explique por que a vazão muda, considerando como a construção esponja absorve a mensagem em blocos de r bits.
- (b) Calcule a vazão aproximada da nova configuração, assumindo que o custo de cada invocação de Keccak- f é o mesmo nos dois casos.

Exercício 7

Queremos calcular o hash da mensagem curta de dois bytes 0xCCCC usando o SHA-3 como substituto do SHA-2 com 256 bits de saída (SHA3-256, $r = 1088$). Lembre que o SHA-3 anexa o sufixo 01 à mensagem e, em seguida, aplica o *multi-rate padding* 10^*1 até completar um múltiplo de r bits.

(Livro, Problema 11.12)

- (a) Qual é a mensagem após sufixo e padding? Apresente a resposta em notação binária, indicando claramente cada campo (mensagem, sufixo, padding) e a quantidade de zeros.
- (b) Quantos blocos de r bits serão absorvidos pela esponja? Quantas invocações de Keccak- f ocorrem na fase de absorção?

Exercício 8

Considere um estado A do SHA-3 (Keccak- f com $b = 1600$, palavras de $w = 64$ bits) em que todos os bits valem 0, *exceto* os bits cuja coordenada z é igual a zero, isto é, $A[x, y, 0] = 1$ para todos x, y . Lembre que a etapa θ computa

$$A'[x, y, z] = A[x, y, z] \oplus \bigoplus_{y'=0}^4 A[x-1, y', z] \oplus \bigoplus_{y'=0}^4 A[x+1, y', z-1],$$

com índices x módulo 5 e z módulo 64. (Livro, Problema 11.15)

- (a) Quantos bits do estado valem 1? Onde esses bits estão localizados na representação tridimensional do estado (que nome recebe essa “fatia”)?
- (b) Aplique a etapa θ ao estado A e descreva o novo estado A' (analise os bits com $z = 0$, $z = 1$ e $z \geq 2$).